

Defense-in-Depth Failure Probability, Worked Out by Hand

Independent layers multiply their miss rates — but only if they’re truly independent. We compute the combined miss for a stack, the marginal value of each layer, and the correlated-failure trap.

What this document does. Defense in depth works because for an attack to land, *every* layer must miss it. If layers fail independently, the combined miss probability is the *product* of the individual miss rates — shrinking fast. This document computes that product for a layered MCP stack, shows the diminishing marginal value of each added layer, and derives why correlated failures (the injection no filter sees) break the whole calculation.

Concepts covered: per-layer catch/miss rates · independent combined miss $\prod f_i$ · marginal layer value · diminishing returns · correlated failure · why a layer must add new coverage.

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: a stack of catching layers

An attack reaches the backend only if it slips past *every* layer. Let layer i catch an attack with probability c_i , so it *misses* with $f_i = 1 - c_i$. The defense stack:

layer	catch c_i	miss f_i
input validation	0.90	0.10
schema allowlist	0.80	0.20
OPA policy	0.85	0.15
process sandbox	0.70	0.30
network egress allowlist	0.75	0.25

Intuition

The rule that keeps a stack honest: *every layer’s threat model is “all the layers above me already failed.”* Process isolation assumes input validation was bypassed; network policy assumes the process escaped. Because each layer only matters when the others miss, the right way to combine them is to multiply miss probabilities — not add catch rates.

2 Step 1 — Independent combined miss

If failures are independent, an attack penetrates all layers with probability

$$F = \prod_i f_i = 0.10 \times 0.20 \times 0.15 \times 0.30 \times 0.25 = 0.0002250.$$

So the stack catches $1 - F = 0.99978$ of attacks — a 99.98% catch rate built from layers none of which exceeds 90% alone.

Mini-example — this operation in isolation

Five mediocre layers beat one good layer. The best single layer here catches 90% (miss 0.10). Stack all five and the miss drops to 0.000225 — roughly 440× better than the best layer alone. Depth, not any one wall, is what gets you the extra “nines.”

3 Step 2 — The marginal value of a layer

Adding a layer with miss f multiplies the surviving miss by f — so it removes a *fraction* $(1 - f)$ of whatever still gets through. Build the stack up one layer at a time:

after adding	cumulative miss F	catch $1 - F$
input validation	0.1000	90.00%
+ schema allowlist	0.0200	98.00%
+ OPA policy	0.0030	99.70%
+ process sandbox	0.0009	99.91%
+ network egress	0.000225	99.98%

Each layer helps, but the *absolute* gain shrinks: the first layer removed 90%, the fifth removed only another 0.07%. Returns diminish because there is less and less left to catch.

4 Step 3 — The correlated-failure trap

The product rule assumes *independence*. If a single attack class defeats several layers at once — an indirect prompt injection that no text filter recognises — those layers fail *together*, and their misses don’t multiply. Suppose the injection bypasses the three text-inspecting layers (input validation, schema, OPA) as one correlated event with miss $f_{\text{corr}} = 0.30$, leaving only sandbox and network independent:

$$F_{\text{real}} = f_{\text{corr}} \times 0.30 \times 0.25 = 0.30 \times 0.075 = 0.0225.$$

That is 100× worse than the 0.000225 the independent model promised. The “defense in depth” was mostly one layer wearing three hats.

Watch out

Multiplying miss rates is only valid for layers that fail for *different* reasons. Three regex filters are not three layers — they are one layer counted thrice, and an input that beats one beats all. A real layer must catch a failure mode the others *cannot*: that is why the stack mixes process isolation, policy, network, and human-in-the-loop rather than five flavours of input check.

5 The argument, at a glance

#	Idea	Formula	Value
1	Layer miss	$f_i = 1 - c_i$	—
1	Combined miss (indep.)	$\prod f_i$	0.000225
1	Combined catch	$1 - \prod f_i$	99.98%
2	Marginal effect	$\times f$ each	diminishing
3	Correlated miss	$f_{\text{corr}} \prod f_{\text{indep}}$	0.0225 (100× worse)

6 Equation cheat-sheet

Layer miss	$f_i = 1 - c_i$
Combined miss (indep.)	$F = \prod_i f_i$
Combined catch	$1 - F$
Marginal layer effect	multiply surviving miss by f
Correlated failure	shared misses do <i>not</i> multiply

A Reference implementation

```

from math import prod
miss = [0.10, 0.20, 0.15, 0.30, 0.25]      # 1 - catch per layer

F = prod(miss)
print(round(F, 6), round(1 - F, 5))          # 0.000225 0.99978

# correlated: 3 text layers fail together as one event (0.30)
F_real = 0.30 * 0.30 * 0.25
print(round(F_real, 4))                     # 0.0225 (100x worse)

```

— end of document —